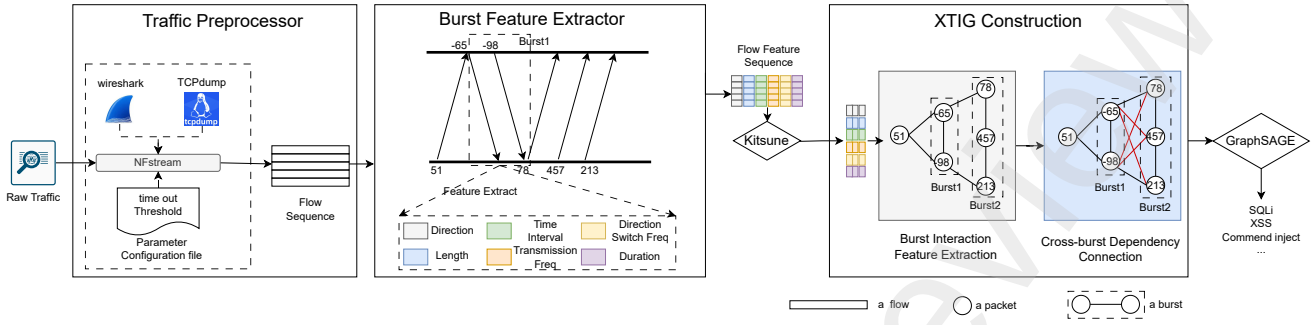
Graphical Abstract

Extended Traffic Interaction Graph: An Efficient Framework for Malicious Encrypted Web Traffic Detection

Wenhao Li, Weidong Zhou, Yuan Zhao, Tianbo Wang, Ying Li, Jian Jiao



Extended Traffic Interaction Graph: An Efficient Framework for Malicious Encrypted Web Traffic Detection

Wenhao Li[a], Weidong Zhou[b], Yuan Zhao[c], Tianbo Wang[d], Ying Li[e] and Jian Jiao[a,*]

[a]School of Computer Science, Beijing Information Science and Technology University, Beijing, 102206, China
[b]Beijing Key Laboratory of Satellite Network Application Technology, China Satellite Network Application Co., Ltd., Beijing, 100160, China
[c]Beijing Key Laboratory of Network Technology, Beihang University, Beijing, 100191, China
[d]School of Cyber Science and Technology, Beihang University, Beijing, 100191, China
[e]Shenzhen Enrigin Technology Co., Ltd., Shenzhen, China

ARTICLE INFO

Keywords:
Malicious encrypted traffic detection
Extended traffic interaction graph
Burst features
GraphSAGE

ABSTRACT

Detecting malicious encrypted Web traffic through communication behavior modeling has become a mainstream research paradigm. A burst is defined as a sequence of packets transmitted consecutively in the same direction. Under encryption, bursts serve as the smallest behavioral units that still preserve interaction semantics. However, existing burst-based modeling methods remain limited. For behavior entity modeling, current approaches often ignore fine-grained features within bursts, such as temporal rhythms and sending frequencies. As a result, the intrinsic temporal characteristics of bursts cannot be effectively captured. For interaction modeling, existing methods assume that dependencies between bursts are only sequential and are conveyed through the start and end packets. This assumption limits the representation of multi-dimensional cross-burst dependencies. To address these problems, we propose a Web malicious encrypted traffic detection framework based on an EXtended Traffic Interaction Graph (XTIG). First, temporal statistical features including burst duration, sending frequency, and inter-packet intervals are integrated to enhance the discriminability of behavioral entities. Second, a cross-burst fully connected structure is introduced to strengthen packet-level associations between adjacent bursts. In addition, a lightweight anomaly filtering mechanism is adopted to reduce computational overhead. GraphSAGE is employed to perform multi-class encrypted Web attack detection, as it aligns well with the structural characteristics of XTIG. Experimental results show that the proposed framework outperforms existing baselines, achieving an average multi-class accuracy improvement of approximately 3.75%. Moreover, the framework demonstrates efficient detection performance in real high-throughput network environments.

1. Introduction

Web malicious encrypted traffic has become the primary vehicle for cyber-attack activities. From October 2023 to September 2024, a total of 32.1 billion encrypted threat incidents were intercepted, representing a year-over-year increase of 10.3%. Among these, cross-site scripting (XSS) attacks surged by 110.2%. Web exploit activities increased by 35%, with a staggering 87.2% of attacks executed through encrypted channels (Zscaler ThreatLabz, 2023). This trend has amplified the scale, stealth, and complexity of Web attacks, placing heightened demands on existing security detection and analysis technologies.

The core challenge of encrypted Web attacks lies in the invisibility of their payload content, rendering detection methods reliant on plaintext semantic parsing difficult to apply directly. In recent years, deep learning-based encrypted traffic detection methods have emerged as a research hotspot (Zhu et al., 2024; He et al., 2023). These approaches typically model traffic behavior to detect encrypted Web traffic. However, from the perspective of attack behavior representation, existing methods still exhibit two shortcomings: On one hand, modeling traffic behavior entities as packet-level or flow-level bursts loses the key temporal information during the attack process to some extent, making it difficult to fully characterize the dynamic features of Web attacks. On the other hand, the modeling of interactions between traffic entities remains relatively limited, making it difficult to effectively capture the complex dependencies within multi-stage Web attack processes.

In terms of behavioral entity modeling, existing studies abstract network traffic at different granularities, including individual packets (Anyfantis et al., 2025), time windows(Li et al., 2025) and bursts (Wang et al., 2025). Single-packet-based approaches preserve low-level information in its entirety; however, their excessive granularity hinders the formation of stable and discriminative high-level behavioral representations. Time-window-based approaches partition network traffic into fixed temporal intervals, yielding a regular entity structure; nevertheless, these partitions often fail to align with the semantic units of real-world Web interactions. Burst-based approaches treat unidirectional packet sequences as behavioral entities, achieving a moderate level of hierarchy and granularity. A unidirectional packet sequence also preserves relatively complete interaction semantics. However, such approaches often overlook fine-grained behavioral characteristics within bursts, including temporal rhythms, local fluctuations, and packet transmission frequency patterns. As shown in the left panel of Figure 1,

*Corresponding author
2023020629@bistu.edu.cn (W. Li); zhouwd@buaa.edu.cn (W. Zhou); zhaoyuan_@buaa.edu.cn (Y. Zhao); wangtb@buaa.edu.cn (T. Wang); vivian.li@enrigin.com.cn (Y. Li); jiaojian@bistu.edu.cn (J. Jiao)
ORCID(s): 0000-0003-3546-1045 (J. Jiao)

unlike normal traffic exhibiting sparse arrival patterns, automated SQL injection attack traffic often displays pronounced high-frequency temporal bursts. This distinction offers clear differentiation from normal user behavior traffic, yet existing methods have not sufficiently modeled or leveraged this characteristic (Shi et al., 2025; Chen et al., 2025).

In terms of entity interaction modeling, literature (Liu et al., 2024) models interactions between behavioral entities using sequential edges or linear connections. This design only captures the temporal order between adjacent entities. It cannot represent richer relationships, such as causal dependencies or protocol-level correlations. The studies in (Makinde, 2025; Bakhshi and Ghita, 2021) construct only local connections between adjacent traffic bursts. For example, they link the start and end nodes of consecutive bursts or adopt sparse graph structures to describe burst interactions. Such modeling strategies easily overlook latent correlations among non-adjacent bursts. The methods in (Seok and Sohn, 2024; Huoh et al., 2023; Sufrianto et al., 2022) attempt to expand graph structures by introducing mechanisms such as global attention. However, these approaches are still constrained by assumptions of local connectivity or limitations on sequence length. As a result, they fail to fully capture cross-stage interaction dependencies that are common in encrypted Web attacks (Wang et al., 2017).

As illustrated on the right side of Figure 1, in a cross-burst command injection scenario, an attacker injects a malicious payload disguised as a benign command parameter in Burst 1 during the request phase. This allows the attack to bypass server-side security checks. In Burst 2, the server returns sensitive environment variables or the execution results of the injected command. Currently, an increasing number of Web attacks deliberately distribute data injection and data exfiltration across different time periods and interaction rounds. Relying only on burst boundary associations or analyzing the context within a single burst makes it difficult to identify such latent cross-burst correlations. Based on the above analysis, this paper aims to address the following challenges:

- How to characterize packet-level temporal relationships?
- How to construct relationships between cross-burst packets?

In order to address the aforementioned problems, we propose a Web MAlicious Encrypted Traffic Detection Framework (WAD) based on XTIG. In behavioral entity modeling, we introduce temporal features, including timestamps, burst duration, local transmission frequency, and direction switching patterns, into the original traffic interaction graph node representation. These features enable a more comprehensive characterization of the internal temporal rhythm and frequency variations within each burst. In interaction modeling, we introduce a fully connected structure across bursts to address the difficulty of existing methods in modeling cross-burst dependencies. This design explicitly captures long-range dependencies in multi-stage Web attacks and preserves the complete attack behavior chain within the graph structure. Based on the above enhancements, a more discriminative XTIG is constructed. It combines feature enhancement and structural enhancement within the traffic interaction graph, providing input representations that jointly capture structural information and behavioral features for subsequent graph neural network–based classification models. Finally, we employ GraphSAGE, a model with inductive learning capabilities, to classify multi-category Web attacks with XTIG, achieving efficient detection.

We summarize the main contributions of this paper as follows:

- We propose WAD framework. It models traffic behavior using a traffic interaction graph that combines feature enhancement and structural enhancement, enabling efficient detection of multiple types of Web malicious encrypted traffic with good engineering efficiency.
- We introduce an extended traffic interaction graph. It models bursts as traffic behavioral entities and represents each burst by integrating multi-dimensional temporal features. It introduces a fully connected entity interaction construction strategy to model cross-burst dependencies. These dual enhancements significantly improve the temporal and structural representation capabilities for Web malicious encrypted traffic.
- We conduct experiments on four datasets to evaluate the performance of the WAD framework. The experimental results demonstrate that WAD achieves clear advantages in detection performance compared with existing classical methods.

2. WAD Framework

The overall system framework workflow is illustrated in Figure 2, primarily comprising four core modules: traffic preprocessor, burst feature extractor, XTIG construction, and graph classification. These modules collaboratively form a layered processing mechanism, enabling efficient and stable classification of malicious traffic.

Traffic Preprocessor: The framework first preprocesses raw network traffic by continuously monitoring network interfaces to capture packets in real time. Packets are aggregated based on quintuple information, forming an ordered flow sequence.

Burst Feature Extractor: After obtaining the flow sequence, the framework divides each stream into bursts. This module segments continuous interactions into multiple burst fragments based on packet arrival times and directional changes. Within each burst, multidimensional statistical features are extracted using packets as the basic unit. These include direction, length, packet-to-packet time intervals, transmission frequency, and burst duration all temporal characteristics forming a burst-level feature sequence. This stage does not involve graph structure modeling and solely characterizes behavioral patterns within bursts.

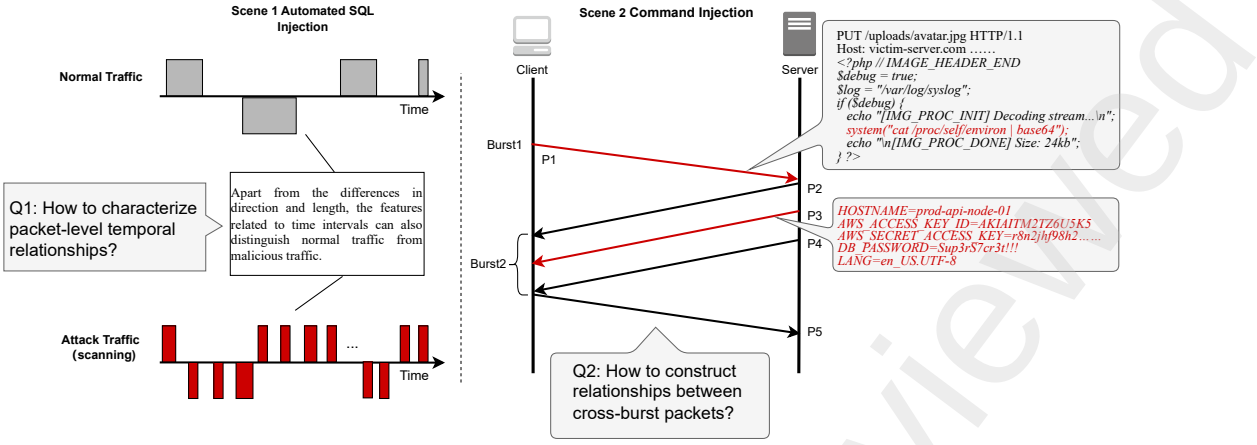


Figure 1: Challenges in Detecting Malicious Encrypted Web Traffic.

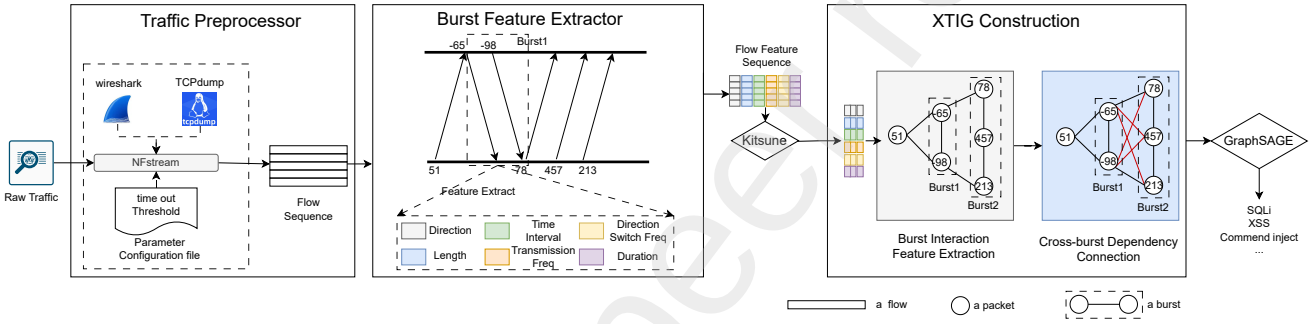


Figure 2: The proposed WAD system framework comprises four main stages: traffic preprocessor, burst feature extractor, XTIG construction and graph classification. Raw traffic is first processed by NFstream into flow sequences; multidimensional burst interaction features are then extracted;Finally, the XTIG is constructed and fed into the GraphSAGE model to detect attack types such as SQL injection and XSS.

XTIG Construction: After burst feature extraction, the system constructs the XTIG. This module preserves the temporal connection relationships between packets within bursts while introducing cross-burst node interaction modeling. It unifies multiple bursts from the same flow into a holistic graph structure. This construction method explicitly captures latent dependencies between bursts, preventing interaction semantics from being fragmented at burst boundaries. This enables a more comprehensive representation of encrypted Web traffic behavior patterns.

Graph Classification: After XTIG construction, each suspicious flow is represented as a structured graph input. GraphSAGE is adopted as the graph classification model. Through neighborhood sampling and multi-layer feature aggregation, structural and behavioral information of nodes is progressively integrated to obtain a graph-level representation of packet interactions, which is then used to classify encrypted malicious Web traffic.

2.1. Traffic Preprocessor

As the data entry point for the WAD framework, the traffic preprocessing module transforms high-throughput raw network data into structured stream-level sequences. This module integrates underlying traffic collection toolchains, directly attaching TCPdump and Wireshark to network interfaces to enable real-time capture and parsing of raw network traffic. Collected data is subsequently fed into the framework's core NFStream flow reconstruction engine for processing.

DFStream maintains a dynamic flow table based on the five-tuple (*srcIP, dstIP, srcPort, dstPort, protocol*). Through a state-tracking mechanism, packets belonging to the same session are aggregated into bidirectional flow sequences. This process hides the complexity caused by packet fragmentation at the data ingestion stage and produces semantically consistent flow-level representations.

In order to adapt to dynamic network environments and improve flow segmentation accuracy, NFStream is centrally controlled by an external parameter configuration file, in which key parameters include the flow timeout threshold. To handle common scenarios such as long-lived connection reuse and abnormal interruptions, the system monitors the inter-arrival time of adjacent packets and applies a timeout-based truncation mechanism to terminate inactive

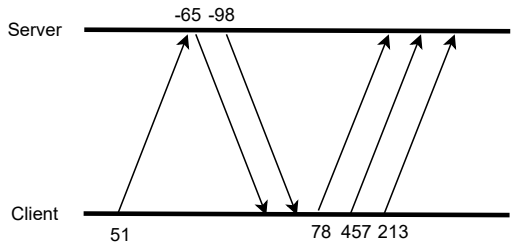


Figure 3: Client-Server Interaction Behavior.

flows in a timely manner. In addition, a head–tail packet detection strategy is introduced to further constrain flow boundaries. This ensures that the resulting flow sequences are structurally complete and semantically consistent, providing high-quality input for subsequent feature extraction and modeling.

2.2. Burst Feature Extractor

This module receives flow sequences from the traffic preprocessor module. It then converts them into a flow feature sequence composed of burst features. Each burst feature consists of two parts: a set of basic burst features and a set of temporal features.

2.2.1. Burst Basic Features

Taking a typical client-server flow sequence as an example (as shown in Figure 3), the communication process can be fundamentally described through the length and direction characteristics of data packets. Specifically, the transmission direction of each packet is first defined and jointly incorporated with the packet length sequence to introduce direction-aware features. As shown in Eq. 1, a flow consisting of N packets can be represented as $F = \{s_n\}$, where each element s_n is defined as the product of the packet length l_n and its corresponding direction coefficient dir_n, i.e., $s_n = l_n \cdot dir_n$.

$$F = \{s_n\} = \{l_n \cdot dir_n\}_{n=1}^{N} \tag{1}$$

$$dir_n = \begin{cases} -1, & \text{if uplink (client to server)} \\ 1, & \text{if downlink (server to client)} \end{cases} \tag{2}$$

Based on the aforementioned fundamental behavioral representations, the module segments burst sequences according to packet transmission direction and temporal continuity. A burst refers to a sequence of packets continuously transmitted by the same communication party within a short timeframe, maintaining consistent transmission direction. Since a burst corresponds to continuous interactions initiated by the same party, no uplink–downlink direction switching occurs within a burst. By using bursts as the basic interaction units, this partitioning strategy organizes the original linear packet sequence into a hierarchical behavioral representation, which is more suitable for characterizing interaction patterns at different stages of communication.

2.2.2. Multidimensional Temporal Features

Although Web encryption technology effectively obscures the content of data payloads, the interaction patterns and temporal sequences during communication cannot be completely concealed. Statistical characteristics at the transport layer and below still reveal the interaction logic between communicating parties. Through in-depth analysis of typical Web attack behaviors, this section identifies two distinct attack characteristics that remain highly distinguishable even in encrypted environments: micro-level burstiness in attack payload timing and macro-level asymmetry in multi-stage interactions. Building upon foundational burst characteristics, this section further introduces targeted timing and interaction enhancement features designed to capture subtle differences in attack traffic across varying granularities.

(1)*Timing burstiness and rhythmic differences of attack payloads.*

Automated Web attack tools (such as SQLMap, DirBuster, Wapiti, etc.) exhibit significant timing burstiness and machine-generated rhythmic patterns when performing vulnerability scanning or fuzz testing. This behavior stems from program logic constraints and the objective of maximizing efficiency. Existing research indicates that unlike the random reading pauses and relatively smooth traffic requests generated by normal users browsing pages, automated tools tend to concentrate high-density requests within short timeframes or exhibit strict periodic pulse characteristics (Thang et al., 2018).

In order to address these characteristics, we introduce the following three statistical features to quantify micro-temporal rhythms at the burst granularity level:

Burst Duration: Let a burst b consist of an ordered packet sequence $\mathcal{P}_b = \{p_1, p_2, \ldots, p_{n_b}\}$, where each packet p_i has a timestamp t_i. The burst duration is defined as the difference between the timestamps of the first and last packets within the burst:

$$D_b = t_{n_b} - t_1 \tag{3}$$

This feature effectively distinguishes brief automated probing behaviors from sustained normal resource loading. Observations from our self-constructed dataset show that scanning-type attacks rapidly traverse targets, resulting in significantly shorter burst durations than normal Web page access involving static resources (e.g., images, scripts).

Transmission Frequency: Based on the burst duration, the transmission frequency is defined as the ratio of the number of packets in a burst to its duration:

$$F_b = \frac{n_b}{D_b} \tag{4}$$

where n_b denotes the number of packets contained in burst b. This metric quantifies the traffic density per unit time. Experiments indicate that traversal attacks and high-frequency injection activities often exhibit abnormally high transmission frequencies. In contrast, normal users, due to pauses caused by human operation and page rendering, demonstrate relatively low and highly fluctuating transmission frequencies.

Time Interval Statistics: To capture fine-grained temporal patterns within a burst, the arrival interval between consecutive packets is defined as:

$$\Delta t_i = t_{i+1} - t_i, \quad i = 1, \ldots, n_b - 1 \tag{5}$$

The mean and variance of these intervals are computed as:

$$\mu_b = \frac{1}{n_b - 1} \sum_{i=1}^{n_b-1} \Delta t_i, \quad \sigma_b^2 = \frac{1}{n_b - 1} \sum_{i=1}^{n_b-1} (\Delta t_i - \mu_b)^2 \tag{6}$$

These statistics capture the inherent temporal fingerprints of automated tools. Prior studies (Thang et al., 2018) indicate that automated scripts typically adopt fixed execution logic or timeout retry mechanisms, leading to packet arrival intervals with low variance. In contrast, normal human-computer interaction traffic, influenced by network fluctuations and user randomness, exhibits significantly greater dispersion in its time interval distribution.

(2)*Asymmetry and Directional Shifts in Multi-Stage Interactions.*

Micro-level timing characteristics alone are insufficient to describe the complete attack process. Complex Web attacks often involve macro-level interaction logic. For instance, SQL blind injection and stored XSS attacks typically comprise multiple logically related interaction stages, exhibiting specific asymmetry and directional shift patterns between requests and responses. Taking SQL blind injection as an example, attackers infer database content through Boolean logic, leading to frequent small-request-small-response bidirectional switching that exhibits intense directional oscillation. In contrast, data exfiltration attacks manifest as sustained unidirectional transmission of small requests paired with large responses (Zhang and Paper, 2019). These interaction pattern differences remain unaffected by encryption protocols and reliably reflect attack intent.

In order to capture this macro-level interaction logic between bursts, we introduce a key interaction metric:

Direction Switching Frequency: Defined as the total number of times packet transmission direction changes (from upstream to downstream or vice versa) within a stream interaction sequence, quantifying the intensity of "dialogue" between communicating parties. Research indicates direction change frequency is a key indicator for classifying encrypted traffic (Al-Naami et al., 2019). Control-based and injection-based attacks require frequent command interactions, exhibiting significantly higher direction switching frequencies than ordinary file downloads or streaming media playback. By incorporating this feature, the model effectively distinguishes complex interactive attacks from simple data transfer activities.

Furthermore, to enhance overall system efficiency, we introduce a lightweight autoencoder cluster based on Kitsune at this stage for unsupervised anomaly screening of traffic, thereby reducing computational overhead in subsequent graph modeling and inference phases.

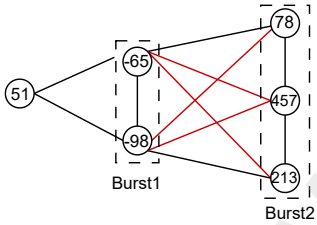


Figure 4: Structure of XTIG.

2.3. XTIG Construction

The construction of the XTIG mainly consists of two steps. First, burst interaction feature extraction models burst interactions as an interaction graph. Second, cross-burst interaction connections are added between intermediate packets of adjacent bursts to enrich cross-burst interaction relationships.

2.3.1. Burst Interaction Feature Extraction

In Web page loading scenarios, traffic exhibits pronounced segmentation and burst characteristics. Significant variations in traffic volume across different information types enable effective differentiation of traffic types based on statistical features and burst patterns. To more precisely characterize interactions within malicious encrypted traffic, this section constructs a graph structure representing inter-burst relationships based on the proposed burst features. Drawing inspiration from Shen's work, the resulting graph is termed the Traffic Interaction Graph (TIG). TIG is a data representation method that transforms discrete network traffic sequences into topological graph structures. Within this structure, each packet in the data stream is typically abstracted as a graph node, with packet attributes such as size and direction mapped as node properties. Simultaneously, edges within the graph connect nodes that are temporally adjacent or logically associated. Through this modeling approach, TIG overcomes the linear constraints of traditional sequential data, explicitly preserving the interactive logic between packets during communication. This enables the capture of complex malicious attack patterns.

2.3.2. Cross-burst Dependency Connection

In order to more concretely characterize interaction behaviors within malicious encrypted traffic, we constructs them into a graph structure representation. Through the form of nodes, edges, and the graph as a whole, the structural relationships within the traffic can be revealed more intuitively. This structure is referred to as the XTIG, and its basic form is illustrated in Figure. 4.

Nodes: To better describe client–server interactions, we construct a graph using individual data packets as nodes. Each node not only contains packet size information but also records the transmission direction and timestamp. Extracting these details as node features facilitates a more comprehensive modeling of bidirectional communication—capturing data sequence structures, length variation trends, and temporal evolution relationships.

Edges: Since sequential packet sequences alone cannot fully describe state transitions between interactions, we classify edges into intra-burst edges and inter-burst edges. For intra-burst edges, adjacent nodes are connected sequentially based on packet timestamps—for example, a sequence may link two packet nodes with a single edge. For inter-burst edges, connections are established between all nodes across adjacent bursts to capture cross-burst interactions.

The algorithm for constructing an expanded traffic interaction graph is shown in Algorithm 1. When interactions between clients and servers are represented solely by packet sequences, it becomes challenging to effectively characterize different types of malicious traffic features, particularly revealing significant shortcomings in modeling burst behavior. Attack traffic often exhibits significant variations in the time intervals and transmission frequencies within bursts, providing critical discriminative information. Therefore, we integrate packet direction, sequence characteristics, and timestamp information during graph construction to build a more discriminative interaction graph structure.

Traditional modeling approaches typically only capture sequential relationships between bursts, assuming attack behavior progresses linearly along the timeline. This fails to reflect the nonlinear characteristics common in encrypted Web attacks, such as multi-round interactions. For instance, in attacks like reflective command injection, malicious payloads are often embedded within normal response streams. Relying solely on connecting the start and end points of adjacent bursts makes it difficult to detect such cross-stage, concealed payloads. By establishing a fully connected structure between adjacent bursts, the model explicitly preserves potential interaction paths between arbitrary burst nodes. This creates tighter semantic associations between request-response behaviors across different phases within the graph structure, preventing fragmentation of behavioral chains due to connection simplification. This structural design aids in characterizing dependency patterns within the multi-stage evolution of Web attacks, providing more comprehensive structural information for subsequent graph-level discrimination.

Algorithm 1 Traffic Interaction Graph Construction

Input: Packet sequence $L = (l_1, \dots, l_n)$, direction sequence $D = (d_1, \dots, d_n)$, timestamp sequence $T = (t_1, \dots, t_n)$
Output: Graph $G = (V, E)$

Initialize node set V and edge set E as empty; initialize burst set B and current burst B_{current} as empty $current_{\text{flag}} \leftarrow$ None
for *each packet* $l_i \in L$ *with direction* $d_i \in D$ *and timestamp* $t_i \in T$ **do**
  if $current_{\text{flag}}$ *is None or* $current_{\text{flag}} = d_i$ **then**
    Add node $v_i = \text{Node}(l_i, d_i, t_i)$ to B_{current} $current_{\text{flag}} \leftarrow d_i$
  end
  else
    Add B_{current} to B Initialize B_{current} and add node $v_i = \text{Node}(l_i, d_i, t_i)$ to B_{current} $current_{\text{flag}} \leftarrow d_i$
  end
end
Add B_{current} to B
for *each burst* $b_i \in B$ **do**
  if $|b_i| > 1$ **then**
    for *each adjacent node pair* $v_k, v_{k+1} \in b_i$ *in order* **do**
      Add edge (v_k, v_{k+1}) to E
    end
  end
end
for *each pair of adjacent bursts* $b_i, b_{i+1} \in B$ **do**
  if $|b_i| = 1$ *and* $|b_{i+1}| = 1$ **then**
    Add edge (v_i, v_{i+1}) to E
  end
  else
    for *each* $v_p \in b_i$ *and* $v_q \in b_{i+1}$ **do**
      Add edge (v_p, v_q) to E
    end
  end
end
Set $V = \bigcup b_i, b_i \in B$
return *Graph* $G = (V, E)$

2.4. Graph Classification

Considering that fully connected strategies may introduce issues with edge growth, this section introduces GraphSAGE's local neighborhood sampling and aggregation mechanism at the model level. This ensures node representations rely solely on a finite-scale sampled neighborhood, effectively suppressing noise propagation and computational overhead caused by redundant edges while preserving key interaction relationships. This design balances structural expressiveness with computational efficiency and model stability in real-world scenarios. To validate the performance of the selected model, this section conducts theoretical analysis and experimental evaluation of four commonly used graph neural network architectures: GCN, GAT, traditional message-passing GNN, and GraphSAGE.

2.4.1. Theoretical Analysis of Model Selection

Regarding graph classification model selection, we systematically evaluate four commonly used graph neural network architectures: GAT, GNN, GCN, and GraphSAGE, comparing dimensions including feature representation capability, computational complexity, real-time inference performance, and inductive generalization ability.

First, analyzing GCN, its node updates are implemented via the following normalized convolution operation:

$$h_v^{(l+1)} = \sigma\left(\sum_{u \in \mathcal{N}(v) \cup \{v\}} \frac{1}{\sqrt{|\mathcal{N}(v)|\,|\mathcal{N}(u)|}} h_u^{(l)} W^{(l)}\right) \quad (7)$$

The model employs a fixed mean normalization aggregation method, featuring a simple structure. However, its

single-layer computational complexity $O(d_v \cdot D \cdot F)$ scales linearly with the node degree d_v. In the XTIG, the formation of locally dense neighborhoods due to intra-burst connections and cross-burst associations results in a significant increase in node degrees at peak positions. Consequently, the inference overhead of GCN escalates drastically with density, making it difficult to satisfy the strict latency requirements of real-time detection scenarios.

Although traditional message-passing GNNs possess stronger expressive capabilities, their general form is given by:

$$h_v^{(l+1)} = \text{UPDATE}\left(h_v^{(l)}, \text{AGG}\{h_u^{(l)} \mid u \in \mathcal{N}(v)\}\right) \quad (8)$$

This approach requires a complete traversal of all neighboring nodes. Consequently, in high-density substructures, it faces computational expansion issues similar to those of GCNs. The inference time is significantly influenced by the scale of the neighborhood, making it difficult to guarantee stable latency in online encrypted traffic environments.

GAT assigns differentiated weights to neighboring nodes through an attention mechanism, with attention coefficients calculated as:

$$\alpha_{vu} = \frac{\exp\left(\text{LeakyReLU}\left(\mathbf{a}^\top [W h_v \| W h_u]\right)\right)}{\sum_{k \in \mathcal{N}(v)} \exp\left(e_{vk}\right)} \quad (9)$$

This mechanism enables a more fine-grained characterization of critical interaction patterns within bursts. However, the calculation of attention weights requires evaluating all neighbor pairs, resulting in a complexity approaching $O(d_v^2 \cdot F)$ in practical implementations. In the locally dense regions frequently appearing in the XTIG, the inference cost rises steeply, making it challenging to maintain real-time performance in high-speed encrypted traffic scenarios.

After a comprehensive comparison, GraphSAGE demonstrates overall advantages that are better suited to the XTIG structure. GraphSAGE adopts an inductive learning framework and introduces fixed-size neighborhood sampling. Its node update method is defined as:

$$h_v^{(l+1)} = \sigma\left(W^{(l)} \cdot \text{AGG}\{h_u^{(l)} \mid u \in S_{\mathcal{N}(v)}\}\right) \quad (10)$$

where $S_{\mathcal{N}(v)}$ represents a fixed number of nodes sampled from the neighborhood ($|S| = k$). This sampling mechanism results in a computational complexity of [insert formula], which is decoupled from the actual neighborhood size. This fundamentally avoids the computational explosion problem faced by GCN, conventional GNNs, and GAT in high-density neighborhoods. Meanwhile, through optional aggregation functions such as mean and max-pooling, GraphSAGE achieves a favorable balance between expressive capability and computational efficiency, effectively adapting to the burst sequence structure and cross-burst dependency patterns within the XTIG.

GraphSAGE achieves robust generalization capabilities through inductive learning. Unlike traditional transductive methods, it learns a function applicable to unseen vertices, generating effective embeddings for these new nodes. This property is particularly crucial in cryptographic traffic scenarios with variable packet counts, while ensuring the model can be extended to diverse graph structures—even interaction graphs not encountered during training. Furthermore, GraphSAGE employs a fixed-size neighbor sampling strategy for local aggregation, focusing solely on a vertex's immediate neighbors. This naturally aligns with the local dependency patterns inherent in network interactions. By capturing key behavioral features while avoiding redundant noise, it provides a theoretical foundation for efficient and accurate multi-class Web attack classification.

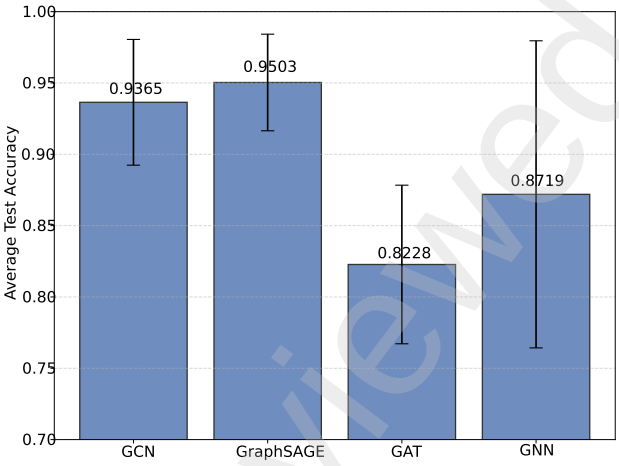


Figure 5: Performance Comparison of Models.

2.4.2. *Experimental Evaluation of Model Selection*

In order to validate the selected model's performance, we evaluate different models using the same dataset. As shown in Figure 5, GraphSAGE achieves optimal classification performance while ensuring minimal inference latency, significantly outperforming GCN, GAT, and traditional GNN models. Considering computational efficiency, structural adaptability, inductive generalization capability, and system deployment feasibility, GraphSAGE is ultimately selected as the core graph neural network model for the WAD framework.

2.5. Theoretical Complexity Analysis

This section conducts a theoretical complexity analysis of the proposed method, covering both time and space complexity, to demonstrate the computational efficiency of WAD.

2.5.1. *Time Complexity Analysis*

In the data preprocessing stage, the algorithm performs feature extraction and graph construction for N raw traffic samples. Since the flow sequence length of each sample is truncated to a constant M, and both burst segmentation and feature computation are implemented via a single linear scan, the time complexity of this stage is $O(N \cdot M)$. During the model training and inference stages, benefiting from

the efficient aggregation mechanism of the GraphSAGE operator, the computational cost of a single convolution layer scales linearly with the graph size, i.e., $O(|V| + |E|)$. As the preprocessing step ensures that the number of nodes in each graph satisfies $|V| \leq M$ and the graph structure is sparse, the inference latency per sample remains low and constant. The overall time complexity of the training process is $O\left(E_{\text{poch}} \cdot N\right)$, where E_{poch} denotes the number of training epochs.

2.5.2. Space Complexity Analysis

The space overhead of the proposed framework mainly consists of two components: dataset storage and runtime GPU memory consumption. In terms of data storage, an in-memory dataset construction strategy is adopted, where the graph structures of all N samples (including a 23-dimensional node feature matrix and adjacency lists) are loaded into memory at once. The corresponding space complexity is $O(N \cdot M \cdot F)$, where F denotes the feature dimensionality. During model execution, the designed lightweight graph neural network contains a small number of parameters, and the GPU memory consumption in the inference stage scales linearly with the batch size, i.e., $O(\text{Batch} \cdot M \cdot F)$.

3. Experimental Evaluation

This section conducts systematic experiments and ablation studies across four datasets—including one self-constructed dataset and three public benchmark datasets—to validate the effectiveness of the WAD framework. First, Section 4.1-4.3 introduces the experimental setup, including baseline methods, dataset details, and evaluation metrics. Subsequently, Section 4.4 evaluates the framework's overall performance and operational efficiency through comparative experiments with existing mainstream methods and engineering complexity analysis. Finally, to validate the superiority of key submodules within the framework, Section 4.5 conducts in-depth ablation experiments analyzing the feature enhancement strategy and graph structure improvement mechanism.

3.1. Baseline Methods

In order to verify the effectiveness of the proposed method, five representative encrypted traffic classification methods were selected as baseline models, covering mainstream approaches such as statistical feature modeling, deep learning, pre-training, and reconstruction enhancement. A brief overview of each method follows:

GraphDApp(Shen et al., 2021b): A method that constructs traffic interaction graphs based on basic features such as packet length and flow direction, utilizing graph neural networks to classify encrypted traffic.

ET-BERT(Lin et al., 2022): This method introduces Masked BURST and Same-origin BURST prediction tasks within a pre-training framework. It leverages the Transformer-based ET-BERT model to achieve encrypted traffic classification, aiming to enhance the modeling capability for complex traffic patterns.

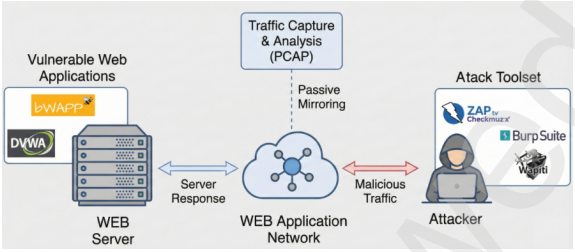


Figure 6: Traffic Preprocessor Architecture of the *Self-constructed Web Attack* Dataset.

FS-Net(Liu et al., 2019): An end-to-end encrypted traffic classification model composed of an encoder, a decoder, a classifier, and a reconstruction layer. It performs classification by learning features from original flow sequences while simultaneously utilizing a reconstruction mechanism to approximate the original sequence, thereby enhancing feature discriminability.

DeepPacket(Lotfollahi et al., 2020): This method uses an end-to-end deep learning model to automatically extract features from individual data packets to achieve encrypted traffic classification.

Fast-DistilBERT(Park et al., 2024): A multi-task learning model based on DistilBERT that simultaneously accomplishes encapsulation type, category, and application classification for encrypted traffic.

The above methods represent different feature modeling and learning paradigms, providing comprehensive baseline references for the performance comparison and analysis of the proposed method.

3.2. Experimental Environment and Datasets

In order to comprehensively evaluate WAD's detection capabilities in encrypted traffic environments, we conduct experimental assessments using four representative encrypted attack scenarios. These include three publicly available cybersecurity datasets USTC-TFC2016, CICIDS2017, and CICMalAnal2017 alongside a self-constructed Web attack dataset.

Self-constructed Web Attack: Addressing limitations in existing encrypted Web attack datasets regarding attack category coverage and traffic organization, we construct a dataset tailored for multi-category detection tasks. In the experimental environment, two typical Web vulnerability targets BWAPP and DVWA were deployed on the Web server side and operated under HTTPS encrypted communication. Subsequently, multiple mainstream automated attack and fuzz testing tools (including Burp Suite, Wapiti, and OWASP ZAP) were used to launch attacks against the server, simulating real-world Web attack behaviors. The experimental architecture is shown in Figure 6.

In order to ensure the accuracy of sample annotation and the interpretability of attack behaviors, each traffic stream was restricted to correspond to only one attack type during

Table 1
Distribution of Samples

Attack Type	Sample Count	Percentage
CSRF	2,360	22.25%
Command Injection	2,154	20.31%
SQL Injection	2,150	20.27%
Directory Traversal	1,566	14.77%
File Upload	1,194	11.26%
XSS	1,182	11.14%
Total	10,606	100.00%

collection. Under these conditions, six common Web attack behaviors were implemented: Cross-Site Scripting (XSS), SQL Injection (SQLi), Command Injection, Cross-Site Request Forgery (CSRF), Directory Traversal, and File Upload attacks. The number of traffic samples corresponding to each attack type is shown in Table 3.

USTC-TFC2016(Wang et al., 2017): Released by the University of Science and Technology of China, this dataset primarily supports deep learning-based network traffic classification and anomaly detection research. This dataset contains diverse malware traffic collected from real network environments between 2011 and 2015, including Cridex, Geodo, Htbot, Miuref, and Neris, covering a wide range of communication patterns.

CICIDS2017(Sharafaldin et al., 2018): Released by the Canadian Institute for Cybersecurity in 2017, this serves as one of the benchmark datasets for intrusion detection system research. It encompasses typical attack scenarios including FTP/SSH brute-force attacks, Denial-of-Service (DoS) attacks, Heartbleed exploitations, and Web attacks. Timestamp information preserves the temporal characteristics of attack behaviors.

CICMalAnal2017(Habibi Lashkari et al., 2018): Primarily focuses on analyzing malware communication behavior, encompassing multiple malware families such as adware and ransomware. Benign traffic is generated by the B-Profile system simulation, enabling a relatively realistic reflection of network communication characteristics in complex business environments.

To ensure comparability across different datasets and to highlight the goal of encrypted traffic detection, all raw PCAP files are uniformly preprocessed. Specifically, packet filtering is applied to remove traffic unrelated to encrypted traffic analysis, such as ARP, DNS, and ICMP packets. Duplicate and invalid traffic is discarded. The remaining packets are segmented into flows, while the original five-tuple information (source IP, destination IP, source port, destination port, and protocol) is preserved to support subsequent feature extraction and detection model training.

3.3. Evaluation Metrics

In this paper, we employ four metrics to evaluate performance: Accuracy (ACC), Precision (Pre), Recall (Recall), and F1 score (F1), calculated as follows:

$$\mathrm{ACC} = \frac{TP + TN}{FP + FN + TP + TN} \tag{11}$$

$$\mathrm{Pre} = \frac{TP}{TP + FP} \tag{12}$$

$$\mathrm{Recall} = \frac{TP}{TP + FN} \tag{13}$$

$$\mathrm{F1} = \frac{2 \times Pre \times Recall}{Pre + Recall} \tag{14}$$

3.4. Overall Performance

This section aims to establish the model configuration and evaluate its overall performance. First, key hyperparameters are determined through experiments to obtain their optimal values. Then, the detection performance of the model is evaluated using multiple metrics, including accuracy and recall. In addition, a complexity analysis is conducted from an engineering efficiency perspective to examine the time overhead and resource consumption of the model in real-time traffic processing.

3.4.1. Hyperparameter Analysis

The purpose of the hyperparameter analysis experiment is to examine the impact of graph scale on the system's classification performance and computational overhead, thereby providing parameter selection guidelines for WAD's practical deployment in real-time detection scenarios. The analysis focuses on the trade-off between classification accuracy and processing latency when varying the maximum number of nodes per traffic interaction graph, aiming to verify that the system can meet real-time computational efficiency requirements while maintaining detection accuracy.

We conduct comparative analysis of model performance under different node upper limit configurations. Experimental evaluations demonstrate that the number of nodes in the traffic interaction graph significantly impacts both model performance and computational overhead. As the number of nodes increases, the graph structure captures richer interaction information, leading to an upward trend in model classification accuracy. However, the time costs associated with graph construction, feature extraction, and model training also increase significantly, primarily due to the higher computational burden introduced by more complex graph structures.

As shown in Figure 7, when the number of nodes is small, the model's classification performance is relatively low due to limitations in the graph structure's expressive power. As the number of nodes gradually increases, the improvement in classification accuracy flattens while computational overhead continues to grow. Balancing classification accuracy and processing efficiency, we ultimately set the upper limit for nodes in each traffic interaction graph to 40. This configuration ensures high recognition accuracy while effectively controlling computational complexity, meeting the stringent processing latency requirements of real-time detection scenarios.

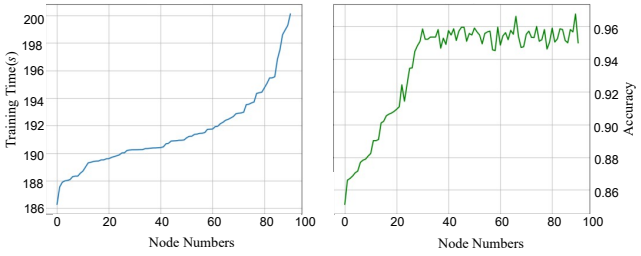


Figure 7: Impact of Node Number on Training Time and Detection Accuracy.

Under this parameter configuration, the model's overall runtime during offline graph construction, feature extraction, and training phases is controlled within hundreds of seconds. The online detection phase involves only graph construction and graph neural network inference, significantly reducing computational overhead. This demonstrates the method's feasibility and lightweight characteristics for practical deployment scenarios.

In order to prevent interference from other model parameters on node count analysis results, we fixed the GraphSAGE network architecture and training configuration during hyperparameter experiments. Specifically, the model employs three layers of GraphSAGE convolutions to extract node features, with channel counts set to 32, 64, and 128 respectively, using ReLU activation functions. Overfitting is mitigated through global average pooling and Dropout (0.025). During training, the Adam optimizer was employed with a learning rate of 0.001, a batch size of 128, and a maximum of 100 training epochs. An early stopping strategy was implemented to prevent overfitting. The cross-entropy loss function was selected, with classification accuracy serving as the evaluation metric. All experiments were conducted on an NVIDIA RTX 2060 GPU using the PyTorch framework, with results based on the optimal model trained on the test set.

3.4.2. Detection Performance

In order to comprehensively validate the effectiveness of the proposed framework in multi-classification tasks for encrypted traffic, we conduct extensive comparative experiments across four datasets: Self-constructed Web Attack, USTC-TFC2016, CICIDS2017, and CICMalAnal2017. All datasets were randomly split into training and testing sets at an 8:2 ratio. Detailed experimental results are shown in Table 2, and the average multi-class classification accuracy improves by approximately 3.75%.

On the Self-constructed Web Attack, WAD demonstrated significant advantages, achieving both accuracy and F1 scores of 97.93%. Compared to the best baseline model ET-BERT (88.48%), our framework improved accuracy by 9.45%, proving its ability to effectively extract fine-grained features from Web attack traffic.

On the USTC-TFC2016 and CICIDS2017 datasets, the framework also delivered outstanding performance, achieving accuracy rates of 90.96% and 94.97%, respectively. These results surpassed the second-best model, Fast-DistilBERT, by approximately 3.61% and 3.00%.

On the CICMalAnal2017 dataset, WAD achieved an average classification accuracy of 92.15%. While its overall performance significantly outperformed most comparison methods like FS-NET and GraphDApp, it fell slightly short of the ET-BERT model (by approximately 1.07%). Further analysis reveals this discrepancy stems from the presence of samples with short durations and sparse packet counts in the dataset. This results in smaller graph structures during the graph construction phase, limiting the comprehensive modeling of inter-node relationships and the global propagation of features. Consequently, the discriminative performance of the graph neural network in this scenario is partially constrained.

In order to further validate the classification accuracy of the WAD framework across different encrypted traffic environments, we analyzed the four datasets involved in the experiment using a confusion matrix heatmap, as shown in Figure 8.

(1) As illustrated in Figure 8(a), the model demonstrates outstanding performance on typical interactive attacks such as command injection, SQL injection, and directory traversal, achieving detection accuracy rates exceeding 99%. This performance advantage primarily stems from the fully connected layers and feature enhancement strategy employed in the XTIG architecture, which effectively captures the behavioral patterns of such attacks during both probing and execution phases. For file upload attacks, 4.19% of samples were misclassified as benign traffic. Further analysis indicates that the encrypted traffic signatures of certain file upload processes closely resemble those of legitimate static resource submissions, leading to local overlap in the graph representation space.

(2) As shown in Figure 8(b), this dataset focuses on distinguishing communication behaviors across different malware families. The proposed method demonstrates strong discriminative power on families with pronounced periodic communication patterns (e.g., Virut and Htbot), achieving accuracies of 97.44% and 95.94%, respectively. The results validate that the temporal features introduced by the model, such as transmission frequency and packet intervals, effectively capture the automated control rhythms of malware. However, within the Shifu family, 12.50% of samples were misclassified as other families, indicating that relying solely on traffic structural features remains challenging when different malware families share similar encrypted HTTP fingerprints or C&C communication patterns.

(3) As shown in Figure 8(c), the model maintains exceptionally high detection rates for various threats including Heartbleed, DoS, and XSS. Brute-force attacks achieved a 91.56% identification rate, though some samples were misclassified as Dropbox traffic. Analysis suggests that for interaction patterns characterized by low frequency and long

Table 2
Evaluation of WAD and baseline frameworks for multi-classification on the Self-constructed Web Attack, USTC-TFC2016, CICIDS2017, and CICMalAnal2017 datasets

Dataset	Baseline	AC(%)	PR(%)	RC(%)	F1(%)
Self-constructed Web Attack	FS-NET	72.78	69.15	66.29	70.27
	GraphDApp	85.56	73.43	71.89	75.13
	ET-BERT	88.48	81.32	80.53	82.12
	DeepPacket	76.41	87.33	75.76	76.89
	Fast-DistilBERT	87.84	86.23	88.26	89.83
	WAD	**97.93**	**97.94**	**96.92**	**97.93**
USTC-TFC2016	FS-NET	78.39	73.79	72.24	76.91
	GraphDApp	82.66	75.11	73.26	77.92
	ET-BERT	85.73	81.88	81.62	82.39
	DeepPacket	79.83	84.34	78.75	79.82
	Fast-DistilBERT	87.35	86.39	82.52	86.13
	WAD	**90.96**	**91.27**	**90.41**	**92.85**
CICIDS2017	FS-NET	74.48	68.94	66.21	70.86
	GraphDApp	82.96	77.31	73.76	75.68
	ET-BERT	82.29	75.85	74.22	76.89
	DeepPacket	84.21	87.13	82.51	75.26
	Fast-DistilBERT	91.97	89.79	90.76	90.96
	WAD	**94.97**	**93.41**	**90.35**	**92.86**
CICMalAnal2017	FS-NET	73.29	71.29	71.85	74.54
	GraphDApp	82.49	74.82	73.56	77.87
	ET-BERT	**93.22**	90.35	89.89	91.12
	DeepPacket	89.21	84.21	89.29	91.85
	Fast-DistilBERT	90.56	89.23	90.33	91.91
	WAD	92.15	**91.47**	**91.52**	**91.53**

intervals, the fixed node upper limit in the graph construction strategy may result in the loss of some long-range contextual information.

(4) As shown in Figure 8(d), the model maintains high stability in detecting ransomware and scareware. However, the detection accuracy for SMSMalware drops to 89.70%. This is primarily because such traffic typically involves a very small number of packets and short interaction durations, resulting in sparse XTIG nodes. This sparsity limits the feature aggregation capability of the GraphSAGE neighborhood sampling mechanism.

3.4.3. Engineering Complexity Analysis

Given the rapid fluctuations of network traffic in real-world environments, high detection latency can delay alarm responses and degrade system availability. Therefore, a hybrid testing environment is constructed for validation. Specifically, encrypted Web attack traffic from the self-constructed Web Attack Dataset is injected into background normal traffic captured in real time from the server network interface at random time intervals. This setup simulates an actual operational scenario under attack.

As shown in Figure 9, the anomaly detection component processed experiments within 1 ms to 0.8 ms, while the multi-classification model averaged between 0.185 s and 0.195 s. Given the minimal proportion of malicious traffic, even higher classification processing times did not cause an overall increase in latency trends.

To further validate the real-time performance of the system, we compared the packet ingress rate under default operating conditions with the maximum processing rate of the proposed framework. As shown in Figure 10, the reported packet reception rate corresponds to a smoothed curve obtained via temporal averaging to reduce fluctuations. Under normal conditions, the packet ingress rate generally hovers around 300 pkt/s, and even during peak periods, it consistently remains below 3000 pkt/s. Meanwhile, the detection framework maintains an average processing rate of approximately 4200 pkt/s. These results demonstrate that the system possesses sufficient processing capacity to meet the requirements of real-time traffic detection.

Simultaneously, we evaluate resource consumption. Figure 11 illustrates the memory usage of processes within the detection framework from startup to entering normal detection state. Experimental results demonstrate that the anomaly detection framework maintains consumption below 2 GB during operation, enabling efficient extraction and classification of interactive behaviors from real-time traffic. To reduce memory usage, we maintain a clocked stream table that promptly clears expired streams. Furthermore, segmented loading and traffic batch processing mechanisms are employed to further control memory peaks while ensuring real-time performance and detection accuracy.

Figure 8: Confusion Matrices of Classification Results on Four Datasets.

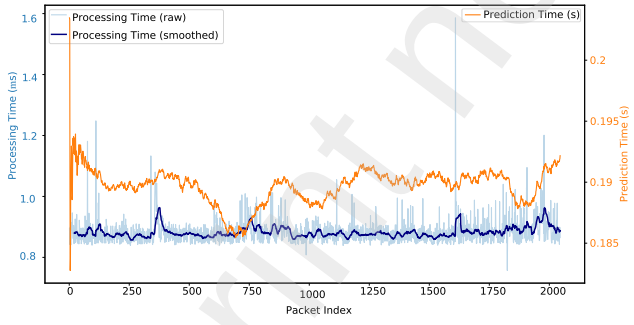


Figure 9: Detection Latency of WAD Framework.

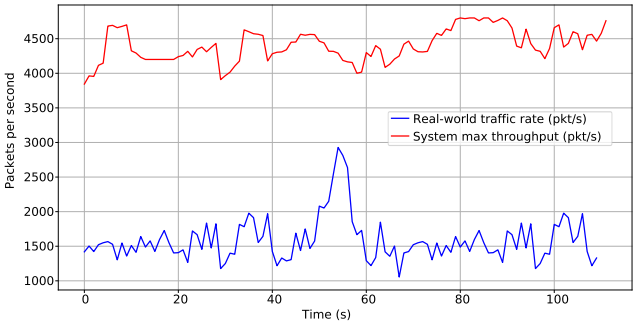


Figure 10: Packet Throughput of WAD Framework.

The real-time classification framework maintains stable processing under sustained high-speed traffic input, with no noticeable data backlog or system congestion. This demonstrates the framework's robust real-time detection capabilities, enabling timely identification and classification of encrypted Web attacks in authentic high-throughput network environments.

3.5. Ablation Studies

In order to validate the effectiveness of the proposed feature enhancement strategy and traffic interaction graph structure improvement mechanism in multi-classification tasks for encrypted Web attacks, this section designs ablation experiments. By removing burst-time features and the

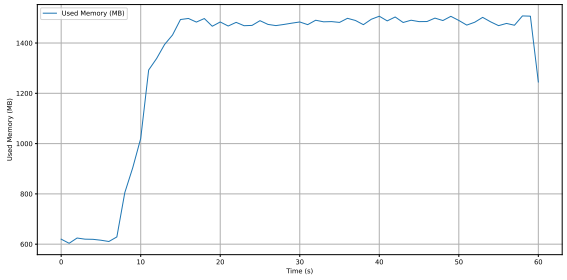
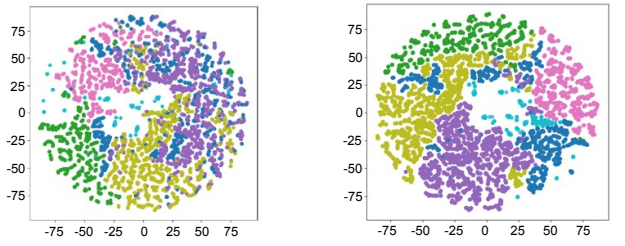


Figure 11: Memory Consumption of WAD Framework.



(a) Original Features (b) Enhanced Temporal Features

Figure 12: t-SNE Distribution Visualization Before and After Feature Enhancement.

cross-burst fully-connected structure, we compare changes in model performance across different configurations.

3.5.1. Ablation Analysis of Feature Enhancement Strategies

In order to evaluate the impact of the introduced burst temporal and frequency features on the discriminative capability of the model, six common types of Web attack traffic together with benign traffic are selected as the test set. We compare the t-SNE (t-distributed Stochastic Neighbor Embedding) visualizations obtained using only basic packet features (packet length and direction) with those obtained after incorporating burst temporal features.

As shown in Figure 12(a), when only packet length sequences and direction features are used, different traffic categories exhibit significant overlap in the low-dimensional space, with blurred boundaries between classes. However, after incorporating temporal features such as burst duration, transmission frequency, and time intervals (as shown in Figure 12(b)), the distribution of attack traffic across categories becomes more concentrated in the embedded space, with significantly clearer inter-class boundaries. This indicates that the added features effectively enhance the separability of the graph representation.

Furthermore, statistical analysis of similarity distributions for identical Web attack types within the XTIG representation space reveals that incorporating temporal features markedly enhances consistency among same-type samples in the embedding space. Distinctiveness between different attack types is significantly improved, further validating the effectiveness of the feature enhancement strategy in capturing fine-grained differences in attack behaviors.

Table 3
Ablation Analysis of Feature Enhancement Strategies

Features	Precision	Recall	F1-score
Basic Features	80.25%	77.38%	74.71%
+Time Interval	93.72%	94.71%	94.39%
+Duration	89.20%	88.45%	86.90%
+Local Frequency	91.15%	90.30%	89.50%
+Directional Frequency	87.85%	86.90%	84.20%
WAD	96.71%	95.98%	96.08%

Table 4
Precision Comparison Before and After Graph Structure Improvement

Method	Precision
Single Connection	93.90%
Full Connection	96.17%

In order to validate the effectiveness of the multi-dimensional temporal features for detecting encrypted Web attacks, we designed feature ablation experiments. Building upon a baseline model retaining only basic features (statistical features), we sequentially introduced four sets of features: direction switching frequency, burst duration, local transmission frequency, and temporal interval statistics. The experimental results are presented in Table 3.

The results show that all feature groups positively contribute to detection performance, with inter-arrival time statistics providing the most significant improvement, indicating that timing regularities induced by machine-driven attack behaviors serve as strong discriminative cues in encrypted traffic. The inclusion of burst duration and local frequency features further enhances performance by capturing traffic intensity and transmission rhythms, effectively distinguishing high-frequency attack bursts from smoother human-driven browsing behaviors and reducing false positives. In addition, incorporating the direction switching frequency increases the F1-score to 84.20%, highlighting interaction density as a key factor for differentiating short request–response attacks from sustained unidirectional flows such as data exfiltration. When all feature groups are jointly integrated, the WAD framework achieves the best overall performance, confirming that temporal, directional, and frequency-related features are complementary rather than redundant, and that their joint modeling yields a more comprehensive representation of encrypted Web attack behaviors.

3.5.2. Ablation Analysis of Graph Structure Improvement Mechanisms

In order to validate the role of cross-burst fully connected structures in modeling multi-stage interaction dependencies of Web attacks, we further compare two graph configurations: the single-connection structure linking adjacent burst heads and tails, and the cross-burst fully connected structure. Under consistent node features, graph scale, and classification models, we conduct comparative analysis of classification performance across different structural configurations.

The ablation results from both the feature and structural perspectives show that combining feature enhancement and structural improvement significantly improves the model's representation ability for encrypted Web attacks.

Specifically, the temporal feature enhancement strategy focuses on fine-grained modeling. By introducing direction-switching statistics and burst-level features, it improves the model's ability to recognize short-period and high-frequency interaction behaviors, such as scanning and probing activities.In contrast, the cross-burst fully connected structure emphasizes global associations. It effectively mitigates the semantic fragmentation problem that occurs in long-period and complex attacks, such as file upload operations.Experimental results confirm that the combination of these two strategies provides graph neural networks with input representations that capture both local details and global context. This multi-dimensional information fusion mechanism alleviates the feature sparsity issue caused by a single perspective and achieves the best detection performance in the final multi-class classification task.

4. Related Work

This section provides a comprehensive review of related studies. It summarizes the current research on behavioral entity modeling and graph structure modeling for malicious encrypted Web traffic addressed in this paper.

4.1. Behavioral Entity Modeling

Existing research on encrypted traffic classification mainly adopts three types of behavioral entity modeling strategies: single-packet–based modeling, time-window–based modeling, and burst-based modeling, where a burst is defined as a sequence of consecutive packets transmitted in the same direction (Wang et al., 2017; Sirinam et al., 2018). Different entity granularities exhibit distinct trade-offs in terms of information preservation, modeling complexity, and robustness to network perturbations.

Early approaches predominantly relied on single packets as the basic modeling unit, focusing on packet-level statistical features such as packet length and transmission direction. Representative methods extract these low-level features to construct traffic fingerprints for encrypted application identification or user activity inference in encrypted environments (Taylor et al., 2016; Li et al., 2022). Although packet-level modeling can preserve fine-grained information, it is highly sensitive to noise and network jitter, making it difficult to robustly capture complete interaction behaviors.

In order to balance computational efficiency and privacy preservation, subsequent studies adopt fixed time windows or flow-level statistical features to model encrypted traffic (Zaki et al., 2022; Lotfollahi et al., 2020). These methods aggregate packet sequences within predefined temporal windows and leverage machine learning or deep learning models to improve classification performance. However, fixed window partitioning is often misaligned with real Web interaction semantics, which may fragment coherent behavioral patterns and weaken semantic consistency.In contrast, burst-based modeling has gradually become the mainstream approach due to its stronger robustness against encryption-induced perturbations and its closer alignment with client–server interaction semantics. Recent works construct behavioral entities by segmenting traffic into directional bursts and perform classification based on burst-level representations (Shen et al., 2021a; Wang et al., 2023). Nevertheless, most existing methods mainly rely on packet length sequences or simple statistical descriptors (e.g., average packet length and direction ratios) when constructing burst-level features, while the internal temporal rhythm of bursts is largely overlooked.However, existing methods mostly rely only on packet length sequences or simple statistics (e.g., average packet length, direction ratio) when constructing node features, paying insufficient attention to the temporal rhythm within the burst. High-discrimination features such as local transmission frequency fluctuations, direction switching counts, burst duration distribution, and relative time intervals are often ignored. These temporal features exhibit significant differences in typical Web attacks such as SQL injection multi-round probing, XSS payload fragmentation, and CSRF rapid verification. However, they have not yet been fully modeled and utilized, resulting in the behavioral rhythm of different attack stages being difficult to express effectively, thereby limiting the separability of multi-class Web attacks. Existing research on encrypted traffic classification mainly adopts three types of behavioral entity modeling strategies: single-packet–based modeling, time-window–based modeling, and burst-based modeling, where a burst is defined as a sequence of consecutive packets transmitted in the same direction (Wang et al., 2017; Sirinam et al., 2018). Different entity granularities exhibit distinct trade-offs in terms of information preservation, modeling complexity, and robustness to network perturbations.

4.2. Graph Structure Modeling

In recent years, graph-based representation learning has shown strong potential for encrypted traffic classification. The key idea of graph-based approaches is to transform traffic data into graph structures and leverage graph representation learning techniques to extract relational features. Owing to their rich structural properties, graphs are well suited for modeling dependencies among traffic instances, where samples are inherently correlated rather than independent. As a result, the independence assumptions commonly adopted by traditional machine learning methods no longer hold for graph-structured traffic data (Wu et al., 2020).

Graph embedding techniques have been demonstrated to be effective in learning low-dimensional representations from graph-structured data while preserving both node attributes and structural information (Li et al., 2020; Zhang et al., 2021). Building upon these advances, recent studies increasingly adopt Graph Neural Networks (GNNs) to model encrypted traffic and perform traffic classification or malicious behavior detection (Jiang et al., 2022; Sun et al., 2020).Existing graph-based encrypted traffic analysis methods construct graphs from communication behaviors and

exploit correlations among traffic flows to improve detection performance. Some approaches integrate flow-level statistical features with graph topology information to identify malicious encrypted traffic, demonstrating that relational modeling can significantly enhance detection capability (Wang et al., 2020; Zheng et al., 2022). Other studies further incorporate temporal or spatiotemporal characteristics into graph representations, enabling more expressive modeling of traffic dynamics and improving classification robustness (Fu et al., 2022).Despite their effectiveness, most existing methods focus primarily on graph topology and coarse-grained flow statistics, while the fine-grained behavioral patterns within encrypted Web interactions remain insufficiently explored. In particular, how to jointly model burst-level temporal dynamics and interaction semantics within a graph framework has not been fully addressed, which limits the applicability of current graph-based methods to fine-grained and multi-class encrypted Web attack classification.

5. Summary

In this paper, we address the challenge of identifying encrypted Web attacks by constructing a Web malicious encrypted traffic detection framework based on an extended traffic interaction graph. The framework treats bursts as traffic behavioral entities, integrating multidimensional temporal rhythm features into entity modeling to enhance expressiveness, while introducing a cross-burst fully connected mechanism in entity interactions to capture long-range dependencies. Experimental results on multiple mainstream public datasets demonstrate that this approach improves detection performance for encrypted Web attack traffic, particularly against automated scanning and complex injection-based Web traffic. Future research will further explore strategies for achieving high-precision classification under data imbalance conditions.

CRediT authorship contribution statement

Wenhao Li: Writing – review & editing, Writing – original draft, Methodology, Investigation, Data curation, Conceptualization. **Weidong Zhou:** Writing – review & editing, Supervision, Methodology. **Yuan Zhao:** Validation, Formal analysis. **Tianbo Wang:** Validation, Formal analysis. **Ying Li:** Validation, Resources. **Jian Jiao:** Writing – review & editing, Validation, Resources, Funding acquisition.

References

K. Al-Naami, S. Chandra, A. Mustafa, et al. Adaptive encrypted traffic fingerprinting with bi-directional dependence. In *Proceedings of the 35th Annual Computer Security Applications Conference (ACSAC)*, pages 177–189, 2019.

G. Anyfantis, I. Castell-Uroz, and P. Barlet-Ros. Graph neural networks for graph-level anomaly detection in network intrusion detection. In *Proceedings of the 9th Network Traffic Measurement and Analysis Conference (TMA)*, pages 1–4. IEEE, 2025.

T. Bakhshi and B. Ghita. Anomaly detection in encrypted internet traffic using hybrid deep learning. *Security and Communication Networks*, page 5363750, 2021.

Z. Chen, X. Wei, and Y. Wang. Encrypted traffic classification encoder based on lightweight graph representation. *Scientific Reports*, 15:28564, 2025.

Z. Fu, M. Liu, Y. Qin, J. Zhang, Y. Zou, Q. Yin, Q. Li, and H. Duan. Encrypted malware traffic detection via graph-based network analysis. In *Proceedings of the 25th International Symposium on Research in Attacks, Intrusions and Defenses (RAID)*, pages 495–509. ACM, 2022.

A. Habibi Lashkari, A. F. A. Kadir, L. Taheri, and A. A. Ghorbani. Toward developing a systematic approach to generate benchmark android malware datasets and classification. In *Proceedings of the IEEE International Carnahan Conference on Security Technology (ICCST)*, 2018.

H. He, Y. Lai, Y. Wang, et al. A data skew-based unknown traffic classification approach for tls applications. *Future Generation Computer Systems*, 138:1–12, 2023.

T. L. Huoh, Y. Luo, P. Li, and T. Zhang. Flow-based encrypted network traffic classification with graph neural networks. *IEEE Transactions on Network and Service Management*, 20(2):1224–1237, 2023.

M. Jiang, Z. Li, P. Fu, et al. Accurate mobile-app fingerprinting using flow-level relationship with graph neural networks. *Computer Networks*, 217: 109309, 2022.

J. Li, S. Wu, H. Zhou, X. Luo, T. Wang, Y. Liu, and X. Ma. Packet-level open-world app fingerprinting on wireless traffic. In *Proceedings of the Network and Distributed System Security Symposium (NDSS)*, 2022.

K. Li, G. Luo, Y. Ye, et al. Adversarial privacy-preserving graph embedding against inference attack. *IEEE Internet of Things Journal*, 8(8):6904–6915, 2020.

Z. Li, H. Zhao, J. Zhao, Y. Jiang, and F. Bu. Sat-net: a staggered attention network using graph neural networks for encrypted traffic classification. *Journal of Network and Computer Applications*, 233:104069, 2025.

X. Lin, G. Xiong, G. Gou, Z. Li, J. Shi, and J. Yu. Et-bert: A contextualized datagram representation with pre-training transformers for encrypted traffic classification. In *Proceedings of the ACM Web Conference (WWW)*, pages 633–642, 2022.

C. Liu, L. He, G. Xiong, Z. Cao, and Z. Li. Fs-net: A flow sequence network for encrypted traffic classification. In *Proceedings of IEEE INFOCOM*, pages 1171–1179, 2019.

M. Liu, Q. Yang, W. Wang, and S. Liu. Semi-supervised encrypted malicious traffic detection based on multimodal traffic characteristics. *Sensors*, 24(20):6507, 2024.

M. Lotfollahi, M. Jafari Siavoshani, R. Shirali Hossein Zade, and M. Saberian. Deep packet: A novel approach for encrypted traffic classification using deep learning. *Soft Computing*, 24(3):1999–2012, 2020.

O. Makinde. Graph neural networks for anomaly detection in encrypted network traffic flows. *International Journal on Science and Technology*, 2025.

J.-T. Park, C.-Y. Shin, U.-J. Baek, and M.-S. Kim. Fast and accurate multi-task learning for encrypted network traffic classification. *Applied Sciences*, 14(7):3073, 2024.

B. Seok and K. Sohn. Adversarial attacks on pre-trained deep learning models for encrypted traffic analysis. *Journal of Web Engineering*, 23 (6):749–768, 2024.

I. Sharafaldin, A. Habibi Lashkari, and A. A. Ghorbani. Toward generating a new intrusion detection dataset and intrusion traffic characterization. In *Proceedings of the 4th International Conference on Information Systems Security and Privacy (ICISSP)*, 2018.

M. Shen, Z. Gao, L. Zhu, and K. Xu. Efficient fine-grained website fingerprinting via encrypted traffic analysis with deep learning. In *Proceedings of the IEEE/ACM International Symposium on Quality of Service (IWQoS)*, 2021a.

M. Shen, J. Zhang, L. Zhu, K. Xu, and X. Du. Accurate decentralized application identification via encrypted traffic analysis using graph neural networks. *IEEE Transactions on Information Forensics and Security*, 16:2367–2380, 2021b.

Q. Shi, D. Shi, L. He, Q. Xiao, L. Zheng, J. Ma, J. He, and Q. Zhang. Transgraphnet: robust detection of malicious encrypted network traffic via transformer and graph neural models. *PeerJ Computer Science*,

2025. DOI: 10.7717/peerj-cs.3353.

P. Sirinam, M. Imani, M. Juarez, and M. Wright. Deep fingerprinting: Undermining website fingerprinting defenses with deep learning. In *Proceedings of the ACM SIGSAC Conference on Computer and Communications Security (CCS)*, 2018.

S. Sufrianto, L. Harudin, A. O. W. Alomairi, and M. A. J. Maktoof. A graph neural network approach for identifying decentralized applications in encrypted traffic. In *Proceedings of the 13th International Conference on Applied Innovations in IT (ICAIIT)*, 2022.

B. Sun, W. Yang, M. Yan, et al. An encrypted traffic classification method combining graph convolutional network and autoencoder. In *Proceedings of the 2020 IEEE 39th International Performance Computing and Communications Conference (IPCCC)*, pages 1–8. IEEE, 2020.

V. F. Taylor, R. Spolaor, M. Conti, and I. Martinovic. Appscanner: Automatic fingerprinting of smartphone apps from encrypted network traffic. In *Proceedings of the IEEE European Symposium on Security and Privacy (EuroS&P)*, 2016.

T. Thang, H. Nguyen, V. Nguyen, et al. Distinguishing between web attacks and vulnerability scans based on behavioral characteristics. In *Proceedings of the 10th International Conference on Management of Digital EcoSystems*, pages 169–176. ACM, 2018.

L. Wang, X. Ma, N. Li, Q. Lv, Y. Wang, W. Huang, and H. Chen. Tgprint: Attack fingerprint classification on encrypted network traffic based graph convolution attention networks. *Computers & Security*, 135: 103466, 2023.

R. Wang, J. Zhao, H. Zhang, L. He, H. Li, and M. Huang. Network traffic analysis based on graph neural networks: a scoping review. *Big Data and Cognitive Computing*, 9(11):270, 2025.

W. Wang, M. Zhu, J. Wang, X. Zeng, and Z. Yang. End-to-end encrypted traffic classification with one-dimensional convolution neural networks. In *Proceedings of the IEEE International Conference on Intelligence and Security Informatics (ISI)*, 2017.

W. Wang, Y. Shang, Y. He, et al. Bot mark: Automated botnet detection with hybrid analysis of flow-based and graph-based traffic behaviors. *Information Sciences*, 511:284–296, 2020.

Z. Wu, S. Pan, F. Chen, et al. A comprehensive survey on graph neural networks. *IEEE Transactions on Neural Networks and Learning Systems*, 32(1):4–24, 2020.

F. Zaki, F. Afifi, S. Abd Razak, A. Gani, and N. B. Anuar. Grain: Granular multi-label encrypted traffic classification using classifier chain. *Computer Networks*, 213:109084, 2022.

H. Zhang and G. Paper. Sdsiot: An sql injection attack detection and stage identification method based on outbound traffic. *IEEE Access*, 7: 163390–163404, 2019.

K. Zhang, Z. Tian, Z. Cai, et al. Link-privacy preserving graph embedding data publication with adversarial learning. *Tsinghua Science and Technology*, 27(2):244–256, 2021.

J. Zheng, Z. Zeng, and T. Feng. Gcn-eta: High-efficiency encrypted malicious traffic detection. *Security and Communication Networks*, pages 1–11, 2022.

P. Zhu, G. Wang, J. He, Y. Dong, and Y. Chang. An encrypted traffic identification method based on multi-scale feature fusion. *Array*, page 100338, 2024.

Zscaler ThreatLabz. 2023 threatlabz state of encrypted attacks report. Online, 2023. URL https://www.zscaler.com/campaign/threatlabz-encrypted-attacks-report.